External Picking Decision Engine

Integration requirements for SAP EWM:
data, APIs and interaction for layered stock removal

ParaTech Platform R&D Team, ParagonCorp

Requirements note, October 2026 · Draft v0.2

Scope. This note specifies what an external *Picking Decision Engine* (PDE) needs from SAP EWM to take over bin, batch and HU determination for outbound picking, group the resulting tasks into warehouse orders and sequence them. It defines the two core APIs (`getInventoryDetails`, `createWOWT`), the supporting data, the interaction pattern, non-functional requirements and a phased plan. The decision logic is the one prototyped in the *Multi-Strategy Simulation Tools* (tabs 6–7) [6].

Contents

1 Executive Summary **2**

2 Scope and System Context **2**

3 Decision Logic **2**

4 Interaction **4**

5 API 1 – `getInventoryDetails` **4**

6 API 2 – `createWOWT` **5**

7 Supporting Data Contracts **7**

8 Exposing the APIs from SAP **8**

9 Non-Functional Requirements **8**

10 Roadmap **9**
10.1 Risks and mitigations 9
10.2 Open points 9

11 Conclusion **10**

1 Executive Summary

Standard EWM determines the source bin with a storage type search sequence and a stock removal rule, then groups warehouse tasks (WTs) into warehouse orders (WOs) with WOCR [1]. The expected logic is a four-key cascade: *UoM layer* → *earliest expiry* → *storage type priority* → *first goods receipt*, applied order by order across a wave. Part of it fits standard Customizing; the cross-order consumption, the HU-specific grouping and the path sequencing are easier to own, test and evolve outside EWM.

Key observations.

- The search sequence alone gets keys 2 and 3 backwards: EWM searches storage type by storage type and stops when it has enough stock, so storage type would beat expiry. Pooling (a storage type group) is required either way [1].
- Inventory alone is not enough. The PDE also needs the *demand* (ODO items), the *rules* (alternative UoM, priorities per quantity classification, WO limits) and a *layout graph* that EWM does not hold.
- Released warehouse APIs are reported as S/4HANA Cloud only (SAP Note 3093108); on-premise, embedded or decentralized EWM needs custom services [4].

Recommendation. Keep EWM as the system of record and executor. Expose two core APIs—`getInventoryDetails` (read quants) and `createWOWT` (write planned WTs/WOs with fixed source data and sequence)—plus two supporting reads and a confirmation feed. Gate go-live with a *shadow-mode* phase in which the PDE plans every wave while EWM still executes its standard result (Section 10).

2 Scope and System Context

In scope	Bin/batch/HU determination for outbound picking; WT grouping into WOs (replacing WOCR for these items); pick sequencing; multi-order waves where later orders see only the stock left by earlier ones.
Out of scope	Putaway, replenishment, physical inventory, packing logic, goods issue posting.
System of record	SAP EWM keeps stock, WT/WO documents, confirmations and postings. The PDE holds no stock of its own.
Fallback	If the PDE does not answer within the timeout, EWM creates WTs/WOs with its standard strategy and WOCR.

Table 1: Scope of the PDE.

3 Decision Logic

For an ODO item i with quantity q_i in base UoM and alternative UoMs $u \in \{\mathrm{PAL}, \mathrm{CAR}, \mathrm{IBX}, \mathrm{PC}\}$ with conversion factors f_u, the layer quantities are obtained greedily, largest UoM first:

$$n_u = \left\lfloor \frac{r_u}{f_u} \right\rfloor, \qquad r_{\mathrm{PAL}} = q_i, \qquad r_{u'} = r_u - n_u f_u \text{ for the next smaller } u'. \tag{1}$$

Within layer u the candidate set is $Q_u = \{\, k : \mathrm{storedUoM}_k = u,\ \mathrm{prio}_u(\mathrm{Stype}_k) \text{ defined},\ a_k > 0 \,\}$, ranked lexicographically by

$$\left(\mathrm{SLED}_k \uparrow,\ \ \mathrm{prio}_u(\mathrm{Stype}_k) \uparrow,\ \ \mathrm{GR}_k \uparrow \right), \tag{2}$$

and consumed in rank order until n_u is covered; the available quantity a_k is reduced before the next order of the wave is planned.

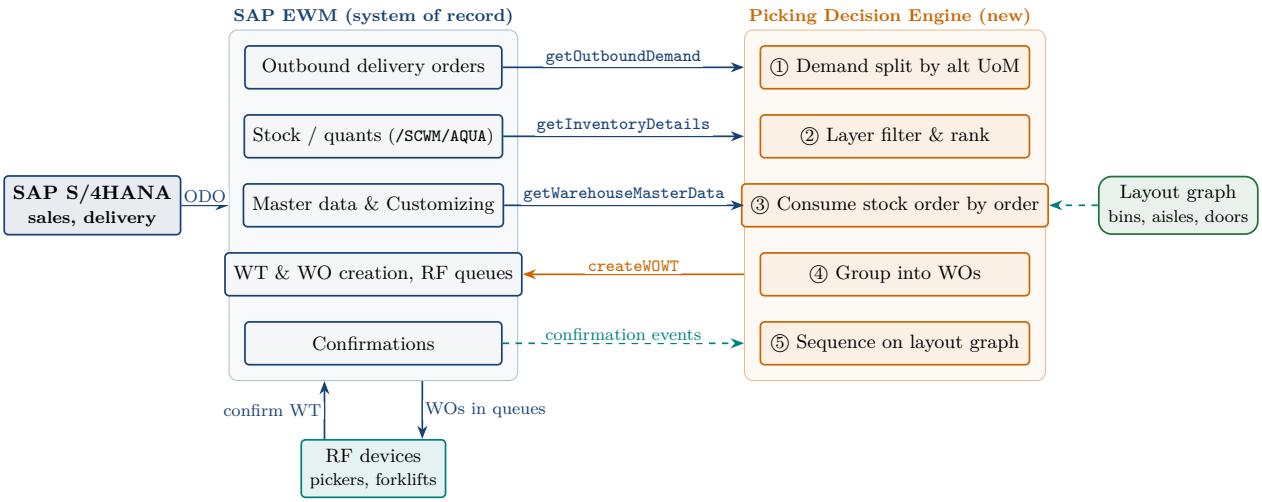


Figure 1: System context. Orange blocks are new engineering. Solid arrows are synchronous calls initiated by the PDE through the integration layer; the orange arrow is the only write into EWM; dashed arrows are events or static data.

1. **UoM layer.** A PAL layer only takes complete pallet HUs; an opened pallet is a CAR quant.
2. **Earliest expiry** (sort field `VFDAT` ascending [3]). Quants below the item's minimum remaining shelf life are excluded first.
3. **Storage type priority** from the search sequence of the layer's quantity classification, all storage types pooled.
4. **First goods receipt** (date and time).
5. **Consumption.** Orders run in a defined sequence (priority, then planned GI time).
6. **Grouping.** Full-pallet HU tasks form a forklift WO; other tasks are grouped per activity area within WO limits (WTs, weight, volume).
7. **Sequencing.** Shortest walk on the layout graph from the start point through all bins of the WO to its staging lane.

4 Interaction

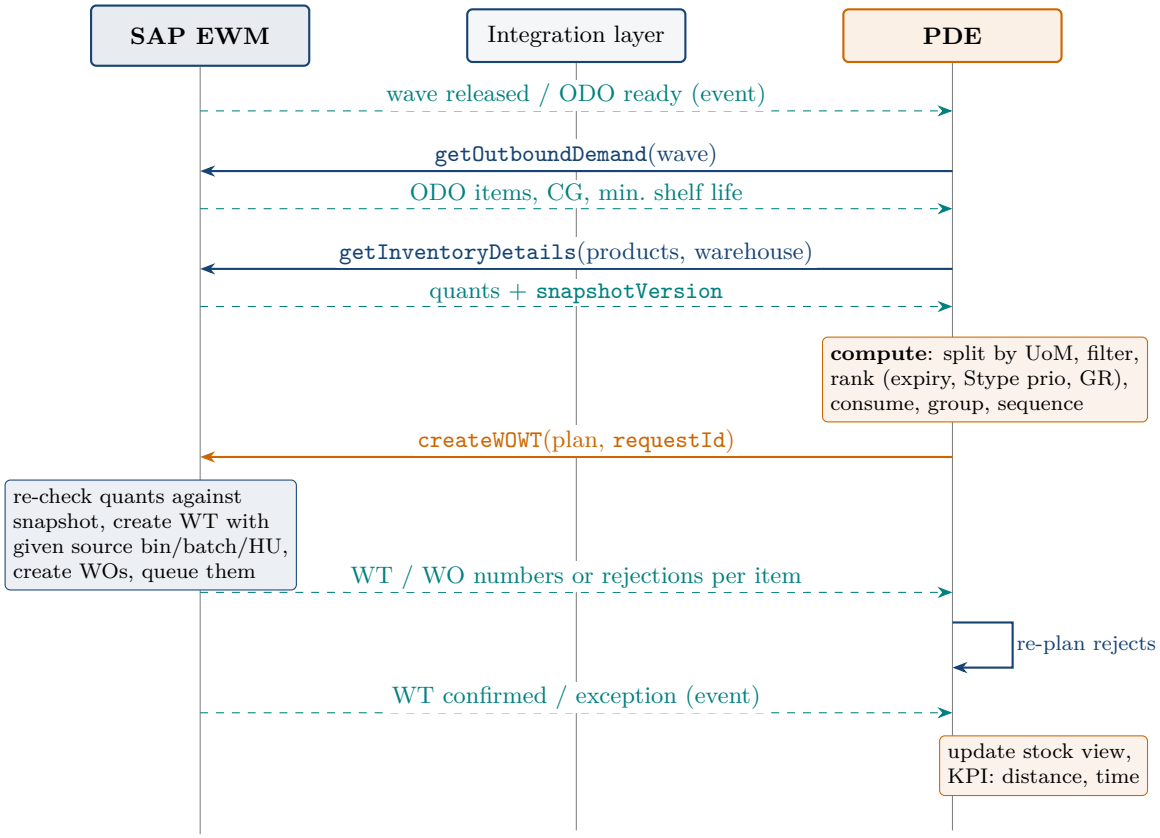


Figure 2: End-to-end interaction for one wave. Blue: reads; orange: the write; dashed teal: responses and events.

5 API 1 – `getInventoryDetails`

Purpose: available stock per quant for the products in the plan. **Call:** `GET`, filtered by warehouse and product list (optionally storage types, stock type). **Volume:** one call per wave, paged. **Freshness:** near real time; the response carries a snapshot version used by `createWOWT`. Table 2 lists the fields, using the EWM structure terms of [2]; Listing 1 shows a response.

Table 2: `getInventoryDetails` response fields (one row per quant).

Field	Type	EWM source *verify*	Used for
`warehouse`	char 4	`LGNUM`	scope
`quantId`	GUID	quant / stock GUID	reference in `createWOWT`
`product`	char 40	`MATNR`	matching demand
`batch`	char 10	`CHARG`	traceability, batch-specific demand
`sled`	date	shelf-life expiration (`VFDAT`)	key 2, min. shelf-life check
`grDateTime`	timestamp	GR date/time (`WDATU`)	key 4
`bin`	char 18	`LGPLA`	source bin, layout lookup
`storageType`	char 4	`LGTYP`	key 3, layer eligibility
`storageSection`	char 4	`LGBER`	reporting, optional rules
`activityArea`	char 4	activity area of the bin	grouping
`qtyAvailable`, `baseUoM`	dec, char 3	physical minus open removal qty	consumption

Field	Type	EWM source *verify*	Used for
`qtyPhysical`, `qtyOpenRemoval`	dec	quant / open WTs	transparency
`storedUoM`, `storedQty`	char 3, dec	stock-specific UoM / packaging	layer filter (PAL, CAR, IBX, PC)
`huId`, `huType`, `isFullHU`	char 20, char 4, bool	HU of the quant	PAL layer, HU WT
`stockType`, `owner`, `entitled`	char	stock category fields	only unrestricted, own stock
`blocked`	bool	bin / quant block indicators	exclude
`changedAt`	timestamp	last change	concurrency check

Envelope: `snapshotVersion`, `generatedAt`, paging token `next`.

```
GET /inventory?warehouse=WH01&products=E,F,G   →  200 OK
```

```
{
  "warehouse": "WH01",
  "snapshotVersion": "2026-10-21T08:15:02Z#4711",          ◁ echo it in createWOWT
  "generatedAt": "2026-10-21T08:15:02Z",
  "next": null,                                            ◁ paging token, null = last page
  "quants": [
    {
      "quantId": "6f1c...",
      "product": "E",
      "batch": "E2611-30",
      "sled": "2026-11-30",                                ◁ key 2: earliest expiry
      "grDateTime": "2026-09-10T07:42:00Z",                ◁ key 4: first goods receipt
      "bin": "06-L9",                                      ◁ source bin, node in layout graph
      "storageType": "0030",                               ◁ key 3: priority per layer
      "storageSection": "0001",
      "activityArea": "PCK-R",                             ◁ WO grouping
      "qtyAvailable": 500,                                 ◁ physical - open removal, base UoM
      "baseUoM": "PC",
      "storedUoM": "PAL",                                  ◁ key 1: UoM layer filter
      "storedQty": 1,
      "huId": "P-1002",
      "huType": "EUR",
      "isFullHU": true,                                    ◁ only full HUs enter the PAL layer
      "stockType": "F2",
      "blocked": false,                                    ◁ true = never a candidate
      "changedAt": "2026-10-20T16:03:11Z"                  ◁ concurrency check
    },
    {
      "quantId": "9a07...",                                ◁ second quant: fields as above
      "product": "E",
      "batch": "E2611-10",
      "sled": "2026-11-10",
      "grDateTime": "2026-08-12T09:10:00Z",
      "bin": "04-R4",
      "storageType": "0020",
      "qtyAvailable": 200,
      "storedUoM": "CAR",
      "storedQty": 4,
      "huId": null,
      "isFullHU": false
    }
  ]
}
```

```
"key"  "string"  123  true / false / null  {} [] :  ,  ◁ annotation, not part of the payload
```

Listing 1: `getInventoryDetails` response excerpt (data from the simulation, tab 7).

6 API 2 – `createWOWT`

Purpose: create the planned WTs with fixed source data and group them into WOs in the given sequence. **Call:** `POST`, idempotent on `requestId`. **Atomicity:** all-or-nothing per WO; other WOs

of the same request may succeed. Table 3 lists the structure; Listings 2 and 3 show a request and its response.

Table 3: `createWOWT` request and response structure.

Request header	
`requestId`, `planRunId`	idempotency key; plan identifier for audit and KPIs
`warehouse`, `snapshotVersion`	scope; inventory version the plan was computed on
Per warehouse order	
`woRef`, `woType`	external id; `PICK` or `FULL_HU`
`queue`, `resourceType`	RF queue and resource (picker, forklift)
`activityArea`, `consolidationGroup`	grouping attributes stored on the WO
`destination`	staging bin / lane / door
`plannedDistance`, `plannedDuration`	KPIs from the PDE
Per warehouse task (ordered list inside the WO)	
`wtRef`, `seqNo`	external id; pick sequence in the WO
`odo`, `odoItem`	warehouse request reference
`product`, `batch`, `qty`, `uom`	what to pick, in the layer UoM (e.g. 2 CAR)
`sourceBin`, `sourceHU`, `quantId`	fixed source; `sourceHU` makes it an HU WT
`processType`, `destinationBin`	warehouse process type (outbound pick ^{verify}) and staging bin
`layerTrace`	UoM layer and deciding key, for audit
Response	
per WO	EWM WO number, or the reject code of the first failing WT
per WT	EWM WT number, or one of `STOCK_CHANGED`, `QTY_INSUFFICIENT`, `BIN_BLOCKED`, `ITEM_ALREADY_PLANNED`, `UOM_INVALID`, `AUTH`

`POST /warehouse-orders  Idempotency-Key: 8f2c4e1a-...`

```
{
  "requestId": "8f2c4e1a-...",                          ◁ idempotency key
  "planRunId": "PDE-20261021-0815-07",
  "warehouse": "WH01",
  "snapshotVersion": "2026-10-21T08:15:02Z#4711",       ◁ from getInventoryDetails
  "warehouseOrders": [
    {
      "woRef": "PDE-WO-3",
      "woType": "FULL_HU",                              ◁ FULL_HU = forklift WO
      "queue": "FORKLIFT",
      "resourceType": "FL",
      "activityArea": "PCK-R",
      "consolidationGroup": "CG-ST17",
      "destination": "9010-LANE-124",                   ◁ GI staging lane of the delivery
      "plannedDistance": 186,                           ◁ metres on the layout graph
      "plannedDuration": 240,                           ◁ seconds
      "tasks": [
        {
          "wtRef": "PDE-WT-1",
          "seqNo": 1,                                   ◁ pick order inside the WO
          "odo": "PM#124",
          "odoItem": 10,
          "product": "E",
          "batch": "E2611-30",
          "qty": 1,
          "uom": "PAL",                                 ◁ layer UoM, not base UoM
          "sourceBin": "06-L9",
          "sourceHU": "P-1002",                         ◁ set = HU WT, pallet not opened
          "quantId": "6f1c...",                         ◁ re-checked against the snapshot
          "destinationBin": "9010-LANE-124",
```

```
          "layerTrace": "PAL; tie on expiry with 09-L2; decided at Stype prio"  ◁ audit: why this quant
        }
      ]
    }
  ]
}
"key"  "string"  123  true / false / null  {} [] :  ,  ◁ annotation, not part of the payload
```

Listing 2: `createWOWT` request excerpt: the forklift WO of delivery PM#124.

```
200 OK   (partial success:  WOs are all-or-nothing)
{
  "requestId": "8f2c4e1a-...",
  "warehouseOrders": [
    {
      "woRef": "PDE-WO-3",
      "status": "CREATED",
      "ewmWO": "20004711",                                  ◁ EWM warehouse order number
      "tasks": [
        {
          "wtRef": "PDE-WT-1",
          "status": "CREATED",
          "ewmWT": "100045671"                              ◁ EWM warehouse task number
        }
      ]
    },
    {
      "woRef": "PDE-WO-2",
      "status": "REJECTED",                                 ◁ whole WO rolled back
      "tasks": [
        {
          "wtRef": "PDE-WT-7",
          "status": "REJECTED",
          "reason": "STOCK_CHANGED",                        ◁ PDE re-plans this item only
          "detail": "04-R4: available 2 CAR, planned 3 CAR"
        }
      ]
    }
  ]
}
"key"  "string"  123  true / false / null  {} [] :  ,  ◁ annotation, not part of the payload
```

Listing 3: `createWOWT` response: one WO created, one rejected because its stock changed after the snapshot.

7 Supporting Data Contracts

The two core APIs are not enough on their own: the PDE also needs the demand, the rules and the layout (Table 4).

Table 4: Supporting data contracts.

Source	Feed	Cadence
`getOutboundDemand`	ODO number and item, product, quantity in base UoM, requested batch, min. remaining shelf life, consolidation group, ship-to, route/door, planned GI time, priority, wave	per wave
`getWarehouseMasterData`	alternative UoM conversions and quantity classification per UoM; batch and shelf-life flags; storage types with role, full-HU removal and mixed-storage flags; search sequence and priority per quantity classification; WO limits per activity area; queues and resources	daily + on change
Layout graph	per bin: x/y (or aisle–stack–level), access point on the aisle, bin type; aisle segments, one-way and forklift-only restrictions; start points, staging lanes, doors. Not standard EWM data: maintained once and versioned	on change

Source	Feed	Cadence
Confirmation events	WT confirmed (actual quantity, actual bin, exceptions), WO completed, stock changes outside the plan	streaming

8 Exposing the APIs from SAP

	SAP S/4HANA Cloud	On-premise / embedded or decentralized EWM
Read stock	released OData APIs (warehouse stock, storage bin) *verify*	custom OData/RFC on the quant view (`/SCWM/AQUA`)
Read demand	warehouse outbound delivery order API *verify*	custom service on ODO items
Create WT/WO	warehouse order and task API *verify*; check that creation with a fixed source is supported	custom service: create WTs with explicit source bin, batch and HU, then build WOs in the given sequence; trigger via PPF action or BAdI before WOCR
Events	SAP Event Mesh / business events *verify*	custom event on WT confirmation

Table 5: Implementation options by deployment model.

A community thread cites SAP Note 3093108: the released warehouse APIs are meant for S/4HANA Cloud and are not supported for embedded or decentralized EWM [4]. A comparable external pick optimizer hooks in after WT creation and leaves the write-back to the customer project [5]. Confirm the deployment model first; for on-premise, plan for custom services.

9 Non-Functional Requirements

Requirement	Specification	Acceptance
Latency	one wave (≤ 500 items) planned and written back within 10 s	p95 $\leq$ 10 s in load test
Fallback	timeout or error → EWM standard strategy and WOCR	no wave left unplanned
Consistency	`createWOWT` re-checks each quant against the snapshot; rejects only the affected WT/WO; PDE re-plans those items	zero double allocations
Idempotency	a repeated `requestId` returns the first result and creates nothing new	replay test passes
Security	OAuth 2.0 client credentials, technical user restricted to the warehouse, TLS	security review signed off
Audit	store `planRunId`, inputs, ranking trace and document numbers for 90 days	any WT traceable to its rank
Monitoring	success, reject and fallback rates; planned vs. actual pick distance	dashboard live before pilot

Table 6: Non-functional requirements and acceptance criteria.

10 Roadmap

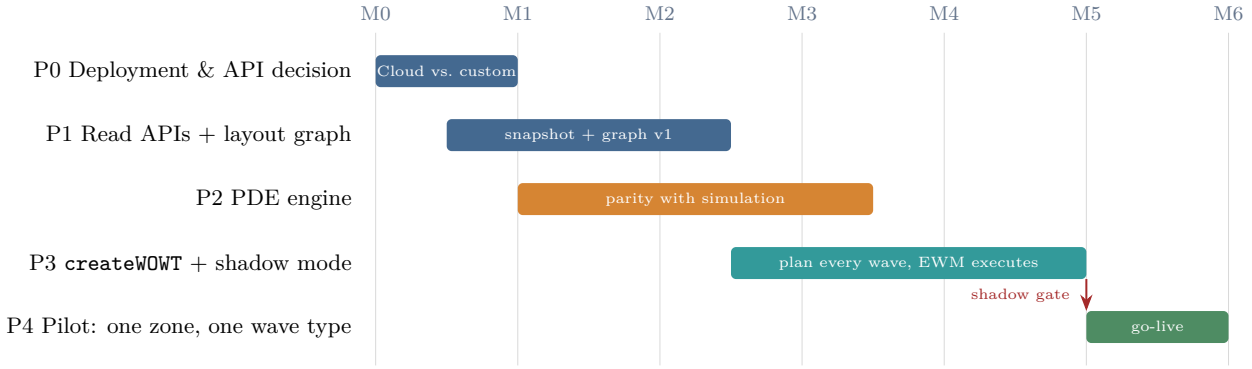


Figure 3: Indicative phasing (months). The shadow gate: for four weeks of real waves the PDE plan must be feasible ($\geq 99\,\%$ of WTs accepted) and must not lengthen the pick distance against the EWM standard result before it is allowed to write.

Phase	Deliverable	Exit criterion
P0	Deployment model confirmed; standard vs. custom API decision; field mapping signed off	Table 5 closed
P1	`getInventoryDetails`, `getOutboundDemand`, `getWarehouseMasterData`; layout graph v1	Snapshot matches EWM stock overview for one day
P2	PDE engine: layers, ranking, consumption, grouping, sequencing	Same picks as the simulation for the tab 6–7 test cases [6]
P3	`createWOWT`, confirmation feed, shadow mode	Shadow gate passed (Fig. 3)
P4	Pilot in one activity area and one wave type	Pick distance and FEFO compliance at or above baseline

Table 7: Phases and exit criteria.

10.1 Risks and mitigations

- **No standard write API for fixed-source WTs** – decide in P0; budget a custom service wrapping WT creation.
- **Stock changes between read and write** – snapshot version plus per-quant re-check; reject and re-plan only the affected items.
- **Stored UoM missing on quants** – derive it from HU type and quantity in P1, or activate stock-specific UoM.
- **Layout graph drifts from the floor** – version it, validate every bin in EWM exists in the graph, re-check after re-slotting.
- **Two sources of truth for rules** – priorities and UoM data are read from EWM Customizing, never retyped in the PDE.

10.2 Open points

- Deployment model (S/4HANA Cloud, embedded or decentralized EWM).
- Can EWM create WTs with a fixed source quant and keep the external WO grouping and sequence unchanged?
- Owner of the layout graph and of the storage type priorities per quantity classification.
- Behaviour when a layer runs short: next smaller UoM, or shortage on the ODO item.

11 Conclusion

The layered logic is small and deterministic; what makes it an integration project is the data around it. With two core APIs, two supporting reads, a versioned layout graph and a shadow gate before the first write, EWM keeps control of stock and execution while the PDE owns the decision—testable against the simulation and replaceable without touching EWM Customizing.

Source note. The decision logic and the sample data come from the Multi-Strategy Simulation Tools and the companion logic note [6]. API names marked *verify* and the EWM field mapping are proposals to be confirmed against the target system; they are not tested interfaces.

References

[1] SAP Learning, "Applying the stock removal strategies," *Basic Customizing in SAP S/4HANA EWM*. [Online]. Available: `https://learning.sap.com/courses/basic-customizing-in-sap-s-4hana-ewm/applying-the-stock-removal-strategies`

[2] SAP Learning, "Outlining warehouse structures," *Processes in SAP S/4HANA EWM*. [Online]. Available: `https://learning.sap.com/courses/processes-in-sap-s-4hana-ewm/outlining-warehouse-structures`

[3] SAP Community, "EWM stock removal strategies," Q&A, Feb. 2016. [Online]. Available: `https://community.sap.com/t5/supply-chain-management-q-a/ewm-stock-removal-strategies/qaq-p/11464648`

[4] SAP Community, "Warehouse order API usage for mobile app," Q&A citing SAP Note 3093108, Nov. 2023. [Online]. Available: `https://community.sap.com/t5/technology-q-a/warehouse-order-api-usage-for-mobile-app/qaq-p/12767484`

[5] Optioryx, "Pulse Pick: SAP EWM integration," documentation. [Online]. Available: `https://docs.optioryx.com/pulse-pick/integrations/sap-ewm`

[6] ParaTech Platform R&D, "Multi-Strategy Simulation Tools" (`ewm-wocr-animation.html`) and "Layered stock removal logic – SAP EWM" (`ewm-layered-stock-removal-logic.md`), internal, Oct. 2026.